

Group members



Ahammad, Jony



Kumara, Muditha



Podkorytov, Nikolai



Shrestha Bata, Prabesh

Your answer

NOT SUBMITTED

Assignment



4 file(s)

Part 0: Tutorial Exploration & Reflection

Purpose

Before starting the technical work, each group must first **study and explore the provided Jupyter Notebook tutorial on Deep Learning Frameworks (TensorFlow/Keras and PyTorch)**.

The purpose of this activity is to help you:

- understand how modern deep learning frameworks work,
- observe how models are built and trained step-by-step,
- learn the meaning and impact of important hyperparameters,
- and reflect on differences between TensorFlow/Keras and PyTorch.

This step ensures everyone has a **shared conceptual understanding** before implementing their own environment and training pipeline.

Instructions

Work in your assigned **group** and carefully go through the **entire Jupyter Notebook tutorial** provided in the course materials.

While exploring the notebook:

- Run the code cells.

- Observe the outputs and figures.
- Modify parameters if appropriate.
- Discuss what you see with your group.

Group Reflection Report

Each group must submit a **short report (2–4 pages)** including the following sections.

1. Summary of What You Learned

Explain in your own words:

- What deep learning frameworks are and why they are useful.
- The roles of **TensorFlow/Keras** and **PyTorch**.
- The key stages of a deep learning workflow:
 - data preparation
 - model definition
 - training
 - evaluation
 - visualization.

2. Key Concepts from the Notebook

Explain the following concepts based on what you observed in the notebook:

- Hidden layers and neural network architecture
- Activation functions (e.g., ReLU, sigmoid, softmax)
- Loss functions
- Optimizers (e.g., Adam, SGD)
- Learning rate

- Batch size
- Epochs
- Dropout / regularization

For each concept briefly explain:

- what it does,
- how it affects training,
- what happens if it is too large or too small.

3. Comparison of Frameworks

Describe at least **three differences** you noticed between:

TensorFlow/Keras and PyTorch

Examples you might discuss:

- code structure
- model definition style
- training loop differences
- debugging and flexibility
- ecosystem tools

4. Screenshots and Observations

Include **screenshots from the notebook**, for example:

- dataset visualization
- training curves
- decision boundary plots
- experiment comparisons

Explain **what each screenshot shows** and what you learned from it.

5. Additional Research

Find **at least two external sources** (documentation, tutorials, articles, or research blogs) and summarize:

- something new you learned,
- how it relates to the notebook tutorial.

Examples of sources:

- TensorFlow documentation
- PyTorch documentation
- research blogs
- educational tutorials.

6. Individual Reflections (for each group member separately)

Reflect briefly on:

- what concepts were most difficult to understand for you,
- what parts of the notebook were most useful,
- what questions you still have about deep learning frameworks.

Deliverable for Part 0

Submit a **PDF report** containing:

- group member names
- explanations of key concepts
- framework comparison
- screenshots and observations
- additional research references

- individual reflections

Part 1: Environment Setup & Mini Project

Establishing a reliable, reproducible deep learning environment

1. Overview and Purpose

In this first part of the assignment, you will learn how to **install** a deep learning framework (TensorFlow or PyTorch), **configure** your environment (using virtual environments or Docker), **verify GPU availability**, and **run a minimal model**. The goal is to ensure that your setup is ready for subsequent, more advanced tasks in deep learning.

2. Step-by-Step Guide

Step 1: Installing a DL Framework

- **Choose a Framework:** Decide whether you want to use **TensorFlow** or **PyTorch** (optionally both) based on your project preferences.
- **Install via pip or conda:**

TensorFlow

```
pip install tensorflow
```

PyTorch

```
pip install torch torchvision
```

Reference:

- [TensorFlow Installation Guide](#)
- [PyTorch Installation Guide](#)

Step 2: Setting Up a Virtual Environment

- **Why Virtual Envs?:** They help isolate dependencies, preventing conflicts across different projects.

Using venv:

```
python -m venv myenv source myenv/bin/activate  
# (On Windows: myenv\Scripts\activate)
```

(Or) Using Conda:

```
conda create -n myenv python=3.8 conda activate myenv
```

Tip: Keep separate environments for each project to maintain reproducibility.

Step 3: Verifying GPU Availability

TensorFlow:

```
import tensorflow as tf  
gpus = tf.config.list_physical_devices('GPU')  
print("GPUs:", gpus)
```

[TensorFlow GPU Guide](#)

PyTorch:

```
import torch  
print("CUDA Available:", torch.cuda.is_available())
```

[PyTorch CUDA Guide](#)

Troubleshoot: If GPUs are not recognized, check your CUDA and cuDNN installation, plus driver compatibility.

Step 4: Addressing Common Environment Errors

- **Typical Issues:**
 - Incompatible CUDA or cuDNN versions
 - Outdated GPU drivers
 - Missing or misconfigured dependencies

- **Resolution:**
 - Update GPU drivers to the latest stable version.
 - Confirm that the framework version supports your installed CUDA version.

[NVIDIA CUDA Compatibility](#)

Step 5: (Optional) Using Docker for Isolation

- **Why Docker?**
 - Ensures a consistent environment for all collaborators.
 - Minimizes dependency conflicts.
- **Basic Dockerfile:**

Dockerfile:

```
FROM tensorflow/tensorflow:2.9.1-gpu
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "your_script.py"]
```

Reference: [Docker Get Started](#)

Step 6: Best Practices in Version Control & Dependencies

- **Version Control:** Use Git or a similar system to track your code and environment files.
 - **Dependency Files:** Keep a `requirements.txt` or `environment.yml` to maintain exact versions.
 - **Reproducibility:** Document your environment setup steps in a README for you and your teammates.
-

Step 7: Minimal Script—Testing Your Setup

1. Create a **Test Script**: Confirm your framework installation and GPU usage with a small model on random data.

2. Example (TensorFlow):

```
import tensorflow as tf
import numpy as np

# Check GPU
gpus = tf.config.list_physical_devices('GPU')
print("GPUs:", gpus)

# Simple model definition
from tensorflow.keras import layers, models
model = models.Sequential([
    layers.Dense(32, activation='relu', input_shape=(20,)),
    layers.Dense(2, activation='softmax')
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy')
model.summary()

# Dummy data
data = np.random.rand(10, 20)
predictions = model.predict(data)
print("Predictions:", predictions)
```

3. Example (PyTorch):

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

Step 8: Mini Project—Putting It All Together

- **Task:** Build and run a minimal deep learning model to confirm your setup.

- **Steps:**
 1. Set Up Environment: Virtual environment or Docker container.
 2. Install DL Framework: TensorFlow or PyTorch.
 3. Verify GPU: Check GPU recognition.
 4. Implement & Run: A simple model (like the snippet above).
- **Deliverable:** Submit your code, screenshots and a brief report describing:
 - Which framework you used and why
 - Any environment challenges you faced and how you resolved them
 - Screenshots and snippets showing successful model output

Step 9: Advanced Troubleshooting and Optimization

- **For deeper issues**, consult:
 - [TensorFlow GPU Troubleshooting](#)
 - [PyTorch Issues on GitHub](#)
- **Optional:** Optimize your environment with advanced features (e.g., `tf.config.set_memory_growth` or `torch.cuda.set_device`) as relevant.

Part 2: Advanced Tutorial Exploration & Reflection

Exploring Model Architecture and Training Behavior

Purpose

Before starting **Part 2**, each group must explore the **second Jupyter Notebook tutorial**.

This notebook focuses on:

- building a more realistic deep learning model,
- understanding architecture design in more detail,
- testing how parameter changes affect results,

- comparing **TensorFlow/Keras** and **PyTorch**,
- measuring training time,
- and testing the trained model on a new image.

This part helps your group build a stronger understanding of model design and experimentation before creating your own training pipeline.

Instructions

Work in your group and go through the entire second notebook.

While exploring it:

- run the code cells,
 - observe the outputs, figures, and timing results,
 - change parameters where possible,
 - compare different settings,
 - discuss your observations,
 - and test the model on at least **one image from your device** if the notebook supports it.
-

Group Reflection Report

Submit a short report (**2–4 pages**) including the following:

1. What You Learned

Briefly explain:

- what the notebook teaches,
- why deeper models are useful,
- how architecture affects model behavior,
- and how experimentation improves performance.

2. Key Concepts

Explain the following based on the notebook:

- convolution layers
- pooling layers
- dense layers
- activation functions
- dropout
- batch normalization
- optimizer
- learning rate
- batch size
- epochs
- model depth / number of layers

For each, explain:

- what it does,
- why it matters,
- and how it affects training.

3. Parameter Exploration

Choose at least **three parameters** you changed, such as:

- number of layers
- number of filters
- dropout
- activation
- optimizer

- learning rate
- batch size

For each, explain:

- what you changed,
- what happened,
- and what you learned.

4. TensorFlow/Keras vs. PyTorch

Describe at least **three differences** you noticed between the two frameworks, for example:

- model-building style
- training loop structure
- flexibility
- debugging
- ease of use

5. Screenshots and Observations

Include screenshots from the notebook, such as:

- training curves
- architecture summary
- confusion matrix
- predictions
- timing results
- feature visualizations

Briefly explain what each screenshot shows.

6. Test on a New Image

Use the notebook to test the trained model on at least **one external image**.

Include:

- the image,
- the prediction,
- whether it seems correct,
- and what you learned from this test.

7. Additional Research

Use at least **two external sources** and explain:

- what new information you found,
- and how it connects to the notebook.

8. Individual Reflection

Each group member should briefly describe:

- what they found most difficult,
- what helped them most,
- and what questions they still have.

Deliverable

Submit a **PDF report** containing:

- group member names
- summary of learning
- concept explanations
- framework comparison
- screenshots and observations
- parameter experiments

- new image test
- external references
- individual reflections

Part 3: Practical Training Pipeline & Mini Project

Applying your deep learning skills with a structured training pipeline.

1. Overview and Purpose

In this part of the assignment, you'll practically apply the skills you've developed to set up an efficient training pipeline. You will work with:

- Data loading and preprocessing.
- Data augmentation.
- Defining and executing a systematic training loop.
- Integrating monitoring and debugging tools (e.g., TensorBoard).

The goal is to build confidence in managing end-to-end training workflows and debugging models.

2. Step-by-Step Guide

Step 1: Data Loading

- Load your dataset using framework-specific tools:
 - TensorFlow: `tf.data.Dataset`
 - PyTorch: `torch.utils.data.DataLoader`

Practical Tutorials:

- [TensorFlow Data Loading](#)
- [PyTorch Data Loading](#)

Step 2: Data Transforms & Augmentation

- Apply transformations to prepare your data (normalization, resizing).
- Use augmentation to artificially expand your training set (rotation, flipping, cropping).

Examples:

- [TensorFlow ImageDataGenerator](#)
- [PyTorch torchvision Transforms](#)

Step 3: Implementing the Training Loop

Your training loop must perform:

- Forward pass: Predictions.
- Compute loss (CrossEntropyLoss recommended).
- Backward propagation (gradient calculation).
- Update parameters (optimizer step).

Quick References:

- [TensorFlow Training Loop](#)
- [PyTorch Training Loop Tutorial](#)

3. Optional: Callbacks & Logging

- **Recommended:** Use callbacks (TensorFlow: TensorBoard, EarlyStopping) or logging tools like Weights & Biases to track experiments.
- This step is optional but strongly encouraged.

Examples:

- [TensorBoard Setup](#)
- [Weights & Biases \(wandb\)](#)

3. Build and Evaluate Your Model

Task: Implement a practical training pipeline for a moderate dataset (if you don't have use for e.g., MNIST, CIFAR-10, or custom moderate-size dataset).

- **Required:**
 - Load and preprocess your data.
 - Train a simple but multi-layer CNN or other advanced network.
 - Evaluate using accuracy and loss metrics on a validation set.

Optional (recommended):

- Implement callbacks (e.g., early stopping).
- Use visualization/logging tools like TensorBoard or wandb.
- Experiment with data augmentation.

3. Deliverables

Submit a brief report including:

- Description of your data pipeline (loading, preprocessing, augmentation).
- Screenshots of your training loop and validation performance.
- Clearly commented code snippets demonstrating each step.
- Individual reflections on challenges encountered and solutions found.

3. Useful Resources & Tutorials

- [TensorFlow Data Pipeline Guide](#)
- [PyTorch Data Loading Tutorial](#)
- [Keras Data Augmentation](#)
- [PyTorch Torchvision Transforms](#)
- [TensorFlow Custom Training Walkthrough](#)
- [PyTorch Training Loop Example](#)

DL_Building_Training_Models_Tutorial.ipynb

...

DL_frameworks_tutorial.ipynb

...

PyTorch-Example.py

...

Tensor-Keras-Example.py

...

The deadline has passed. You can still submit your answer, but it will be marked as submitted too late.